Содержание

АННОТАЦИЯ……4
ВВЕДЕНИЕ……6
1. Аналитический раздел……...8
1.1 Характеристика предметной области……...……...8
1.2 Анализ текущего процесса……...……9
1.3. Анализ аналогов……...……..10
1.4. Требования к системе……11
1.5. Постановка задачи……..12
2. Проектирование системы..... 14
2.1. Проектирование пользовательских сценариев ……...14
2.2. Проектирование интерфейса……15
2.3. Архитектура приложения……16
2.4. Проектирование базы данных……..17
2.5. Модель состояний заказа ……19
2.6. Проектирование REST API……20
2.7. Выбор технологий……..21
3. Реализация web-приложения..... 22
3.1. Организация репозитория……....22
3.2. Реализация регистрации и авторизации……..22
3.3. Реализация каталога и расчётной стоимости……...23
3.4. Реализация заказа……...25
3.5. Реализация файлов……27
3.6. Реализация менеджерской части……..28
3.7. Реализация администрирования……...31
3.8. Адаптивность и доступность……33
3.9. Обработка ошибок и логирование……...34
4. Тестирование и качество..... 37
4.1. Стратегия тестирования……37
4.2. Таблица тест-кейсов……...37
4.3. Найденные и исправленные дефекты……...40
4.4. Проверка безопасности……..41
4.5. Критерии готовности……41
5. Руководство пользователя……...42
5.1. Запуск системы……...42
5.2. Работа клиента……42
5.3. Работа менеджера……...43
5.4. Работа администратора……..43
5.5. Типовые ошибки пользователя……44
Заключение..... 45
Перечень использованных информационных ресурсов..... 46
Приложение А..... 47
Приложение Б……...48
Приложение В……49
Приложение Г……...50
Приложение Д……..51
Приложение Е……...52

АННОТАЦИЯ

В работе представлено полнофункциональное full-stack web-приложение «Сервис аренды инструмента и оборудования», предназначенное для организации и автоматизации процессов аренды инструментов и оборудования между независимыми владельцами и арендаторами. Разработанная система обеспечивает единую цифровую среду для взаимодействия участников арендных отношений, упрощая поиск необходимого оборудования, оформление заявок и контроль выполнения арендных операций. Функциональные возможности приложения включают публичный каталог товаров, древовидную структуру категорий и характеристик, инструменты поиска и фильтрации, регистрацию и управление пользователями, модерацию карточек товаров, учёт физических экземпляров оборудования, обработку заявок, бронирование, выдачу, возврат и обслуживание. Для каждого критически важного этапа предусмотрены серверные проверки прав доступа и допустимых состояний объектов, обеспечивающие соблюдение установленных бизнес-правил. Состояние оборудования при передаче и возврате фиксируется посредством двусторонних фотоактов, что позволяет документировать возможные изменения и повреждения. Итоговая стоимость аренды рассчитывается автоматически на основании её фактической продолжительности с учётом стартовой стоимости, возможной просрочки и согласованного повреждения оборудования.

Клиентская часть приложения реализована с использованием React и TypeScript, обеспечивающих создание интерактивного пользовательского интерфейса и структурированную организацию программного кода. Серверная часть разработана на языке Python с применением Django, Django REST Framework и Django Channels, что позволяет реализовать основную бизнес-логику, обработку запросов и взаимодействие в реальном времени. Для хранения и управления данными используется реляционная СУБД PostgreSQL, а Redis применяется в качестве инфраструктурного компонента для обмена сообщениями и выполнения

фоновых задач. Инструменты Celery и Celery Beat обеспечивают обработку отложенных и периодических операций, снижая нагрузку на основные процессы приложения. Взаимодействие одностраничного клиентского приложения (SPA) с сервером организовано посредством версионированного REST API и соединений WebSocket. Механизм авторизации основан на стандартных cookie-сессиях Django с использованием CSRF-защиты. Для упрощения развёртывания и обеспечения воспроизводимости программного окружения подготовлена конфигурация Docker Compose, объединяющая frontend, backend, PostgreSQL, Redis, рабочий процесс Celery и планировщик периодических задач.

Качество разработанного решения подтверждено комплексом автоматизированных и ручных проверок, направленных на оценку корректности функционирования основных компонентов системы. Приёмочный набор включает восемнадцать последовательных шагов сквозного пользовательского сценария, тринадцать обязательных автоматизированных сценариев и девятнадцать негативных проверок, позволяющих оценить поведение приложения при некорректных действиях и недопустимых состояниях. Выявленные в ходе приёмочного тестирования дефекты были устранены, после чего проведены повторные проверки соответствующих функциональных возможностей. Особое внимание уделено согласованности бизнес-процессов, корректности обработки пользовательских действий и соблюдению ограничений на серверной стороне. В результате реализовано завершённое учебное web-приложение, демонстрирующее комплексный подход к проектированию и разработке современных информационных систем. В рамках единого программного решения согласованы бизнес-правила, пользовательский интерфейс, механизмы хранения данных, фоновые процессы и средства контроля качества, что обеспечивает целостность архитектуры и последовательность выполнения основных операций аренды оборудования.

ВВЕДЕНИЕ

Аренда оборудования позволяет использовать дорогие или редко требующиеся инструменты без их покупки. Для частного клиента и самостоятельного мастера особенно важны понятный выбор, достоверное описание состояния, прозрачные сроки и предсказуемый расчёт стоимости. Владельцу небольшого каталога необходимы учёт каждого физического экземпляра, очередь заявок, контроль выдачи и возврата, история повреждений и сведения о затратах на обслуживание. При ведении этих операций в разрозненных таблицах, переписке и устных договорённостях повышается риск двойного назначения экземпляра, потери сведений о сроках и неоднозначного определения состояния оборудования.

Актуальность проекта определяется необходимостью объединить публичный поиск предложения и строгий операционный учёт в одной информационной системе. Пользователь должен видеть доступные товары разных владельцев, а менеджер — управлять только собственным каталогом и принимать решения по заявкам. При этом система обязана различать заявку, бронь и фактическую аренду, поскольку эти сущности имеют разные права, гарантии и моменты начала расчёта. Отдельное значение имеют фотоакты: они формируют единый набор материалов обеих сторон и связывают фактическую передачу оборудования с последующим возвратом.

Цель работы — разработать завершённое full-stack web-приложение для публикации предложений аренды оборудования и сопровождения полного пользовательского сценария от поиска товара до завершения возврата.

Для достижения цели решены следующие задачи:

– исследована предметная область аренды оборудования и выделены роли участников процесса;

– сформированы функциональные, нефункциональные и ролевые требования;

– спроектированы пользовательские сценарии, структура экранов и визуальные правила интерфейса;

– разработаны архитектура SPA, серверных приложений, фоновых задач и реляционная модель данных;

– реализованы регистрация, каталог, модерация, заявки, брони, фотоакты, аренда, возврат и обслуживание;

– реализованы сохраняемые уведомления, WebSocket-доставка, аудит событий и аналитика;

– выполнены автоматизированное, ролевое, негативное и сквозное приёмочное тестирование;

– подготовлены воспроизводимый запуск, руководство пользователя и сценарий демонстрации.

Объект исследования — процесс предоставления инструментов и оборудования во временное пользование. Предмет исследования — методы проектирования и программной реализации web-системы, обеспечивающей каталогизацию, ролевое взаимодействие, транзакционные переходы состояний, фотофиксацию, расчёт стоимости и контроль качества. Практическая значимость состоит в том, что приложение объединяет публичную площадку и внутренние рабочие процессы менеджера, а его архитектура пригодна для дальнейшего развития без изменения основного разделения ответственности между frontend и backend.

1. АНАЛИТИЧЕСКИЙ РАЗДЕЛ

1.1 Характеристика предметной области

Предметная область включает двух основных участников площадки: арендатора и менеджера. Арендатор выбирает товар, задаёт крайний срок получения и плановый срок возврата, после чего ожидает решения владельца. Менеджер публикует карточки оборудования, учитывает физические экземпляры, рассматривает заявки, вручную назначает свободный экземпляр и сопровождает его до возврата в доступное состояние либо передачи на обслуживание. Администратор выполняет служебные функции: поддерживает категории и характеристики, модерирует новые карточки, просматривает аудит и агрегированную аналитику.

Центральным публичным объектом является товар — описание модели оборудования с фотографиями, минутной ставкой, категорией, характеристиками и точкой самовывоза. Товар объединяет несколько одинаковых физических экземпляров. Для экземпляра предусмотрены операционные состояния «Свободен», «Забронирован», «Осмотр», «Арендован», «Возврат», «Обслуживание» и «Заморожен». Такое разделение позволяет показывать единое предложение в каталоге и одновременно исключать двойное назначение конкретной единицы оборудования.

Заявка не резервирует имущество. Она выражает намерение арендатора получить один экземпляр до заданного момента X. Бронь создаётся только после ручного решения менеджера и назначения свободного экземпляра. Фактическая аренда начинается после прибытия арендатора, завершения материалов фотоакта выдачи и второго актуального подтверждения общего состояния. При возврате начисление прекращается в момент, когда менеджер подтверждает физическое получение оборудования, а не после окончания осмотра. Благодаря этому стоимость зависит от фактического времени пользования, а оформление возврата не увеличивает сумму.

Работа с оборудованием сопровождается неизменяемой хронологией. В ней фиксируются действия пользователя или системы, время, роль инициатора и безопасное описание перехода. Отдельные записи создаются для заявок, броней, аренды, экземпляров, обслуживания, модерации и финансовых решений. Уведомления сохраняются в базе данных и доставляются в открытую сессию через WebSocket; при отсутствии соединения запись остаётся доступной при следующем входе.



Рисунок 1 – Участники и основные варианты использования системы

1.2 Анализ текущего процесса

В неавтоматизированном процессе владелец оборудования размещает объявления на разных площадках и отдельно ведёт список физических экземпляров. Потенциальный арендатор уточняет наличие, адрес и условия в переписке. Если одновременно поступает несколько обращений, менеджеру приходится вручную сопоставлять сроки и помнить, какой экземпляр уже обещан. Сведения о передаче и возврате обычно сохраняются как набор фотографий в сообщениях, поэтому позднее трудно установить, к какому заказу и моменту они относятся.

Расчёт также подвержен неоднозначности. Планируемый срок часто смешивается с фактическим временем использования, а задержка при осмотре

после возврата может ошибочно восприниматься как продолжение аренды. При повреждении отсутствует единая запись с описанием, суммой и решением арендатора. Обслуживание оборудования ведётся отдельно от истории аренд, из-за чего менеджер не получает цельного представления о затратах на конкретный экземпляр.

«Сервис аренды инструмента и оборудования» переводит процесс в последовательность контролируемых состояний. Каталог отвечает за поиск и описание предложения, заявка — за очередь интереса, бронь — за гарантированное удержание назначенного экземпляра до X, фотоакт — за подтверждение состояния, аренда — за фактическое использование, возврат — за прекращение начисления и итоговый расчёт. Каждая операция доступна только разрешённой роли и повторно проверяется на сервере. В результате один и тот же процесс одинаково понимается интерфейсом, API и базой данных.

1.3 Анализ аналогов

Для определения ожидаемого состава функций сопоставлены три распространённых подхода: универсальная доска объявлений, специализированный сервис проката и локальный пункт аренды. Сравнение выполнялось по возможностям, которые влияют на полный цикл: поиск, учёт конкретных экземпляров, очередь заявок, фотофиксация, расчёт по фактическому времени, обслуживание и разграничение ролей. Аналоги рассматривались как функциональные ориентиры; внутренняя архитектура и визуальные решения не копировались.

Критерий	Доска объявлений	Сервис проката	Локальный пункт	Сервис аренды инструмента и оборудования
Публичный каталог	+	+	Частично	+
Поиск и динамические фильтры	+	+	–/частично	+

Критерий	Доска объявлений	Сервис проката	Локальный пункт	Сервис аренды инструмента и оборудования
Учёт физических экземпляров	–	+	+	+
Очередь заявок	–	Частично	Вручную	+
Двусторонние фотоакты	–	Частично	Вручную	+
История и аудит	Не раскрыто	+	Частично	+
Обслуживание и аналитика	–	Частично	Частично	+

Результат сравнения показывает, что для учебного проекта недостаточно только каталога и формы обращения. Ценность «Сервиса аренды инструмента и оборудования» определяется связностью всех этапов. Пользовательское действие меняет конкретную предметную сущность, создаёт запись истории и при необходимости уведомление, а следующий экран строится на сохранённом серверном состоянии. Такое устройство позволяет продемонстрировать не отдельные страницы, а согласованную информационную систему.

1.4 Требования к системе

Функциональные требования сформированы для гостя, арендатора, менеджера и администратора. Нефункциональные требования определяют безопасность, надёжность, производительность, доступность и воспроизводимость запуска. Основные группы приведены в таблице.

Группа	Требование
Функциональность	Каталог, категории, характеристики, профиль, модерация, заявки, брони, выдача, аренда, возврат, обслуживание, уведомления, аудит и аналитика.
Удобство	Русский интерфейс, ясные статусы, адаптация к 360, 768 и 1366 px, сохранение введённых данных после ошибки, клавиатурное управление.

Группа	Требование
Производительность	Серверная пагинация и фильтрация, индексы по владельцам, статусам, категориям и срокам, загрузка изображений подходящего размера.
Безопасность	Cookie-сессия, CSRF, серверная проверка роли и принадлежности объекта, проверка изображений, секреты вне репозитория.
Надёжность	Транзакционные переходы, идемпотентные фоновые задачи, запрет двойного назначения экземпляра, сохранение уведомлений при сбое WebSocket.
Сопровождаемость	Разделение SPA и backend, миграции, OpenAPI, автоматизированные проверки, Docker Compose и единые правила форматирования кода.

1.5 Постановка задачи

Требуется разработать web-приложение «Сервис аренды инструмента и оборудования» с публичным каталогом оборудования и тремя рабочими ролями. Гость должен искать товары, применять категории и характеристики, просматривать карточки и публичные профили менеджеров. После входа арендатор получает точный адрес, карту и возможность создать заявку. Менеджер управляет собственными товарами и экземплярами, отправляет карточки на модерацию, рассматривает очередь и вручную назначает свободную единицу оборудования.

Система должна обеспечить полный переход от заявки к броне, подтверждению прибытия, двусторонней выдаче, фактической аренде и возврату. Финансовая модель использует зафиксированную минутную ставку, стартовую стоимость десяти минут, округление неполной минуты вверх и двойную цену просроченной минуты. Повреждение вводится менеджером как расчётная сумма и отдельно согласуется арендатором. После возврата экземпляр становится доступным либо передаётся на обслуживание с сохранением стоимости ремонта и контрольного фотоакта.

Backend должен быть единственным источником истины для прав, состояний, формул и доступности. SPA отвечает за отображение, ввод и понятную обратную связь, но не заменяет серверные проверки. Критические операции

выполняются транзакционно, фоновые сроки обрабатываются Celery, уведомления доставляются через WebSocket и остаются сохранёнными в PostgreSQL. Решение запускается единым Docker Compose и предоставляет OpenAPI-документацию для проверки контрактов.

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Проектирование пользовательских сценариев

Основной сценарий арендатора начинается в публичном каталоге. Пользователь вводит поисковый запрос только явным действием, выбирает категорию и типизированные фильтры, открывает карточку и после входа видит точку самовывоза. Форма заявки содержит крайний срок получения и строго более поздний плановый возврат. Созданная заявка получает состояние «Ожидает решения» и находится в очереди до решения менеджера. После назначения экземпляра пользователь видит бронь, уведомление и последние завершённые фотографии состояния этого экземпляра.

Сценарий выдачи начинается после подтверждения прибытия. Арендатор добавляет не менее двух фотографий, менеджер — не менее одной; каждая сторона завершает собственный набор, а затем подтверждает общий фотоакт. Запрос изменений делает неактуальными материалы только адресата и сохраняет уже загруженные файлы. Второе актуальное подтверждение запускает аренду и расчёт. Активная аренда показывает длительность, текущую стоимость и плановый возврат; при просрочке состояние и уведомления обновляются фоновой задачей.

При физическом возврате менеджер останавливает начисление отдельной командой. Каждая сторона добавляет минимум одну фотографию. Менеджер может отменить повышающий коэффициент просрочки или указать повреждение. Если арендатор не согласен с суммой, менеджеру разрешено изменить только её; затем требуется новое решение и повторное подтверждение общего акта. После завершения экземпляр получает состояние «Свободен» либо переводится в состояние «Обслуживание». Менеджер сразу видит ожидающие заявки по ближайшему сроку получения.



Рисунок 2 – Карта разделов web-приложения

2.2 Проектирование интерфейса

Визуальная система строится на светлом нейтральном фоне, белых поверхностях и тёмно-зелёном акценте. Публичная часть использует умеренную плотность: поиск, фильтры, сетка карточек и крупные фотографии. Рабочие кабинеты плотнее, поскольку менеджеру необходимо одновременно видеть сроки, статусы, число свободных экземпляров и доступные действия. Цвет дополняет текст, но не заменяет его: нулевая доступность, просрочка, ошибка и успешное состояние имеют явные подписи.

Страница товара на широком экране разделена на две колонки. Слева последовательно расположены название, галерея, точка самовывоза, характеристики и описание. Справа находятся минутная ставка, подача заявки, сведения о менеджере и наличие. На мобильном экране блоки переходят в одну колонку, а основное действие сохраняет область нажатия не менее 44 × 44 px. Таблицы рабочих разделов преобразуются в карточки или получают локальную прокрутку, не расширяющую всю страницу.

Формы имеют постоянные подписи и сообщения рядом с полем. Состояния загрузки, отсутствия данных, ошибки, успеха и запрета представлены явно.

Фотоакты показывают авторство и этап процесса, поэтому пользователь понимает, чьи материалы завершены и кто должен действовать. Диаграммы аналитики снабжены заголовками, периодами, единицами измерения и табличной альтернативой. Поддерживается видимый клавиатурный фокус и режим уменьшенного движения.

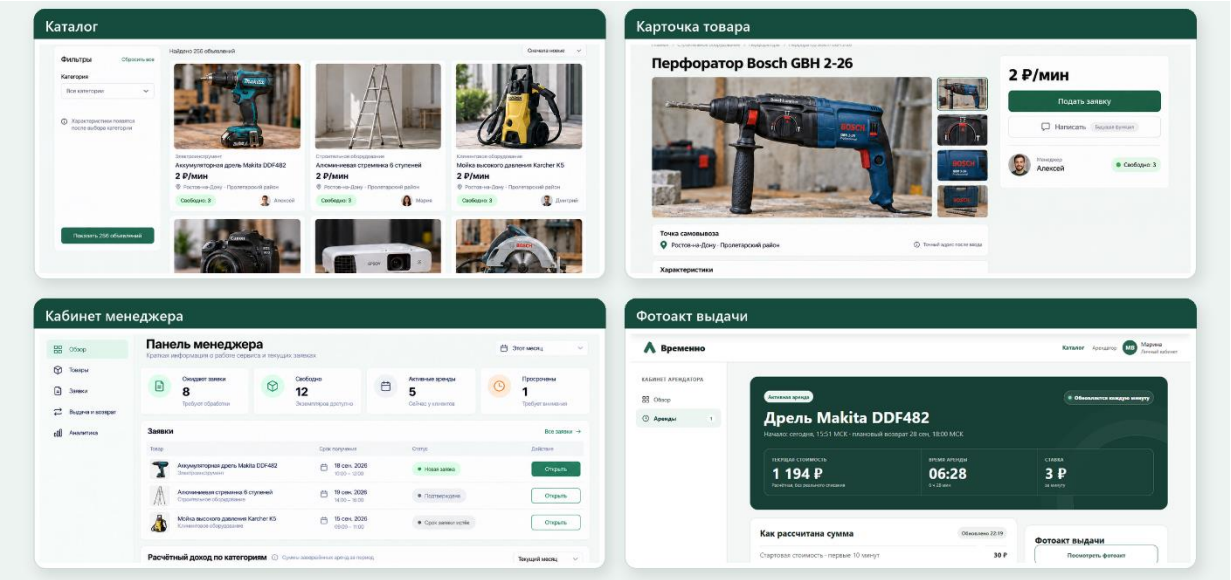


Рисунок 3 – Прототипы ключевых экранов

2.3 Архитектура приложения

Архитектура разделена на отдельное SPA и серверную часть. React-приложение выполняет маршрутизацию, управление формами, клиентскую проверку схем и запросы к API. TanStack Query хранит серверное состояние, повторяет безопасные запросы и обновляет данные после операций. Для критичных экранов выдачи и возврата применяется периодическое обновление, благодаря чему запрос изменений или завершение второй стороны становится видимым без ручной перезагрузки.

Django и Django REST Framework реализуют модели, сериализаторы, сервисные операции и HTTP-контракты. Django Channels обслуживает персональное WebSocket-соединение уведомлений, используя ту же cookie-сессию. PostgreSQL хранит предметные сущности, ограничения и историю; Redis выполняет роль слоя каналов, брокера и хранилища результатов фоновых задач. Celery Beat каждую минуту инициирует обновление истёкших заявок, броней и

просроченных аренд, а рабочий процесс Celery выполняет идемпотентную обработку.

Все многошаговые изменения заключены в транзакции. При назначении экземпляра строки заявки и экземпляра блокируются, после чего повторно проверяются владелец, состояние, срок и наличие существующей брони. Уведомление создаётся в рамках доменной операции, а его WebSocket-доставка планируется через transaction.on_commit. Поэтому сбой канала не отменяет основное действие и не приводит к потере сохранённой записи.



Рисунок 4 – Компонентная архитектура системы

2.4 Проектирование базы данных

Реляционная модель разделяет публичное описание товара и физические экземпляры, а также заявку, бронь и аренду. Категории образуют самоссылочное дерево, характеристики определяются в узлах и наследуются потомками. Значение характеристики товара хранит ссылку на определение и одно типизированное значение. Для модерации, заявок, броней, аренды, обслуживания и уведомлений используются самостоятельные таблицы, что позволяет задавать ограничения БД и вести историю без смешения назначений.



Рисунок 5 – ER-диаграмма базы данных

Таблица	Ключевые поля	Назначение
users_user	id, email, password, name, role, avatar	Учётные записи и роли
categories_category	id, name, parent_id, sort_order	Дерево категорий
categories_characteristic	id, category_id, name, type, required	Определения характеристик
catalog_product	id, manager_id, category_id, status, minute_rate	Карточки товаров
catalog_productinstance	id, product_id, inventory_number, status	Физические экземпляры
catalog_pickuppoint	id, city, district, address, latitude, longitude	Точки самовывоза
applications_rentalapplication	id, renter_id, product_id, status, pickup_deadline_at	Заявки арендаторов
applications_rentalbooking	id, application_id, instance_id, status, minute_rate_snapshot	Брони и снимки условий
applications_handoveract	id, booking_id, completed_at, confirmed_at	Фотоакт выдачи
applications_rental	id, booking_id, status, rental_started_at, return_received_at	Фактическая аренда и расчёт
applications_returnact	id, rental_id, damage_amount, waiver_at	Фотоакт и финансовые решения возврата

Таблица	Ключевые поля	Назначение
catalog_maintenance	id, instance_id, reason, cost, completed_at	Обслуживание экземпляров
notifications_notification	id, recipient_id, event_type, level, read_at	Сохраняемые уведомления
core_auditevent	id, event_type, actor_id, occurred_at, description	Неизменяемый аудит

2.5 Модель состояний заказа

В «Сервисе аренды инструмента и оборудования» основной маршрут состоит из нескольких связанных автоматов состояний. Заявка создаётся как WAITING, может быть отменена или истечь по сроку, а после назначения экземпляра переходит в BOOKED. Бронь имеет состояния ACTIVE, ARRIVED, CANCELLED и EXPIRED. Подтверждение прибытия переводит экземпляр из RESERVED в PICKUP_IN_PROGRESS, но ещё не создаёт аренду. Такое разделение исключает начисление до фактической передачи.

Фотоакт выдачи содержит состояния завершённости и подтверждения для каждой стороны. Изменение материалов сбрасывает только связанные с ними признаки актуальности. Когда обе стороны завершили наборы и подтвердили общий акт, сервер создаёт Rental, фиксирует время начала и переводит экземпляр в RENTED. После планового срока активная аренда становится OVERDUE, но продолжает действовать до физического возврата.

Команда приёма возврата переводит аренду и экземпляр в RETURN_INSPECTION и фиксирует время прекращения начисления. Возврат завершается после актуальных подтверждений обеих сторон и решения по повреждению. Затем аренда становится COMPLETED, а экземпляр — AVAILABLE либо MAINTENANCE. Обслуживание допускается только для свободной единицы, имеет собственную причину, стоимость и контрольные фотографии, после чего экземпляр возвращается в AVAILABLE.



Рисунок 6 – Модель основного жизненного цикла аренды

2.6 Проектирование REST API

API использует префикс /api/v1/, JSON для обычных запросов и multipart/form-data для изображений. Даты передаются в ISO 8601 с часовым поясом. Ошибка содержит машинный код, безопасное сообщение и ошибки конкретных полей. Серверная пагинация применяется к каталогу и рабочим спискам. OpenAPI-схема и Swagger UI доступны в среде разработки и соответствуют фактическим сериализаторам.

Endpoint	Вход	Ответ	Ошибки
POST /auth/register/	email, password, name, role, avatar	CurrentUser	400 validation_error, 403 csrf_failed
GET /products/	search, category, price, filters, ordering, page	Page[Product]	400 параметры
POST /products/{id}/applications/	pickup_deadline_at, planned_return_at	RentalApplication	400, 401, 403
POST /manager/applications/{id}/book/	instance_id	RentalBooking	403, 409 instance_unavailable

Endpoint	Вход	Ответ	Ошибки
POST /bookings/{id}/handover/ confirm/	—	HandoverAct, Rental	400 состояние, 403
POST /manager/rentals/{id}/retu rn/receive/	—	Rental	400 состояние, 403
POST /rentals/{id}/return/finish/	instance_status, maintenance_reason	Rental	400 согласования, 403
GET /manager/analytics/	period или date_from/date_to	ManagerAnalytic s	400 диапазон, 403

2.7 Выбор технологий

React выбран для построения большого числа взаимосвязанных экранов и повторно используемых компонентов. TypeScript в строгом режиме помогает согласовать структуры API, состояния форм и свойства компонентов. Vite обеспечивает быстрый цикл разработки и рабочую сборку, React Router — маршрутизацию, TanStack Query — работу с серверным состоянием, React Hook Form и Zod — управление формами и проверку данных. Material UI предоставляет доступные базовые элементы, которые оформлены собственной темой проекта.

Django предоставляет зрелую модель пользователей, сессии, CSRF, ORM, миграции и административный интерфейс. Django REST Framework используется для сериализации, разрешений и REST API, a drf-spectacular — для OpenAPI. Django Channels и Redis образуют контур уведомлений в реальном времени. Celery и Celery Beat подходят для периодических идемпотентных операций, которые не должны зависеть от открытия конкретной страницы.

PostgreSQL выбран как основная транзакционная база данных благодаря ограничениям целостности, блокировкам строк и эффективной работе со связанными данными. Изображения поддерживают S3-совместимое хранение Yandex Object Storage; локальная среда использует совместимую файловую конфигурацию. Docker Compose обеспечивает единый запуск шести сервисов и одинаковый порядок миграций, загрузки таксономии и создания администратора.

3 РЕАЛИЗАЦИЯ WEB-ПРИЛОЖЕНИЯ

3.1 Организация репозитория

Проект размещён в одном Git-репозитории. Backend разделён на предметные Django-приложения users, categories, catalog, applications, notifications, analytics и core. Frontend организован по функциональным разделам: авторизация, каталог, заявки и аренда, кабинет менеджера, уведомления и аналитика. Документация содержит спецификацию, дизайн-проект, прототипы, журналы сессий и приёмочную трассируемость. Инфраструктурные файлы расположены в корне и позволяют воспроизвести окружение без ручной настройки сервисов.

```
arendator/
├── backend/
│   ├── users/  categories/  catalog/
│   ├── applications/  notifications/  analytics/
│   └── config/  core/  requirements/
├── frontend/
│   └──
src/{api,auth,catalog,applications,manager,notifications,analytic
s}
├── docs/{design,qa,sessions}
├── docker-compose.yml
├── .env.example
├── SPEC.md  DESIGN.md  README.md
└── scripts/
```

Для backend зафиксированы Black, flake8 и mypy, для frontend — ESLint, Vitest, Testing Library, TypeScript и рабочая сборка. Изменения схемы оформляются миграциями. Начальный набор из 73 категорий, 254 характеристик и 370 вариантов загружается идемпотентной командой из YAML, а первоначальный администратор создаётся отдельной командой по переменным окружения.

3.2 Реализация регистрации и авторизации

Пользователь регистрируется как арендатор или менеджер. Email приводится к нижнему регистру и проверяется на уникальность без учёта регистра. Пароль проходит стандартные валидаторы Django и сохраняется только как стойкий хэш. Регистрация автоматически открывает серверную сессию. Текущий пользователь

восстанавливается через GET /auth/me/ после перезагрузки, а выход завершает сессию на сервере и очищает клиентские данные.

Изменяющие запросы защищены CSRF. Cookie имеет HttpOnly, SameSite=Lax и управляемый параметрами окружения Secure. Для отсутствующей или завершённой сессии закрытый API возвращает 401. Роль ADMIN вычисляется для служебного суперпользователя и не предлагается в форме регистрации. Профиль разрешает изменить имя и квадратный аватар; email и роль не входят во входную структуру обновления.

```
class RegistrationSerializer(serializers.ModelSerializer[User]):
    def validate_email(self, value: str) -> str:
        email = User.objects.normalize_email(value).lower()
        if User.objects.filter(email__iexact=email).exists():
            raise serializers.ValidationError(
                "Пользователь с таким email уже
зарегистрирован.")
        return email
    def create(self, validated_data: dict[str, Any]) -> User:
        password = validated_data.pop("password")
        with transaction.atomic():
            return User.objects.create_user(
                password=password, **validated_data)
```

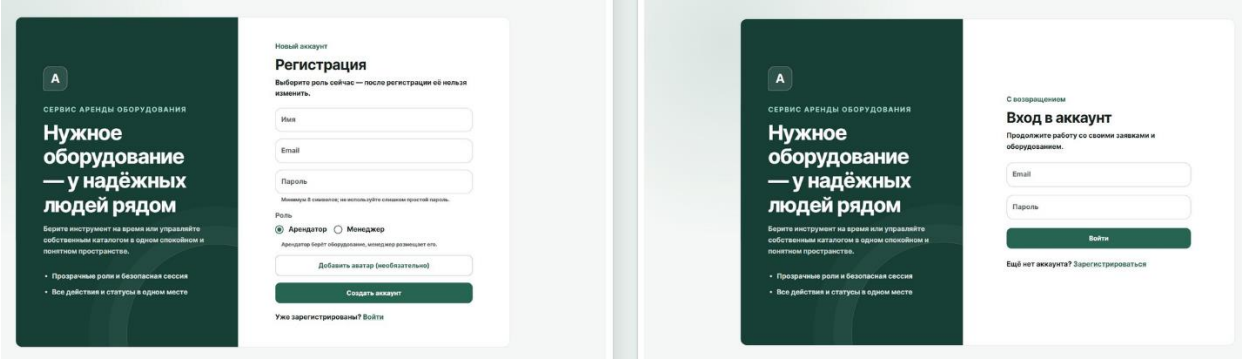


Рисунок 7 – Страницы регистрации и входа

3.3 Реализация каталога и расчётной стоимости

Публичный каталог возвращает только опубликованные товары и использует серверную пагинацию. Поиск выполняется по названию и описанию после кнопки или Enter. Фильтры хранятся в интерфейсе как черновик, предварительный запрос получает фактическое число результатов, а список меняется после применения. Выбор категории включает всех потомков; набор характеристик строится из самой

категории и её предков. LIST принимает несколько вариантов, NUMBER — границы диапазона, BOOLEAN — значение true.

Карточка товара раскрывает гостю город и район, а авторизованному пользователю — полный адрес, координаты и встроенную Yandex Maps. Публичный профиль менеджера разделён на активные товары и архив. Замороженные и скрытые администратором карточки сохраняют историю, но не принимают новые заявки. Минутная ставка отображается в рублях за минуту, а форма заявки рассчитывает ориентировочную длительность между сроками. Окончательные суммы аренды всегда определяет backend по снимку тарифа.

```
def filter_and_order_catalog(queryset, params):
    search = params.get("search", "").strip()
    if search:
        queryset = queryset.filter(
            Q(name__icontains=search) |
            Q(description__icontains=search)
        )
    queryset = apply_price_filter(queryset, params)
    queryset = apply_category_filters(queryset, params)
    ordering = params.get("ordering", "newest")
    return queryset.order_by(*CATALOG_ORDER_FIELDS[ordering])
```

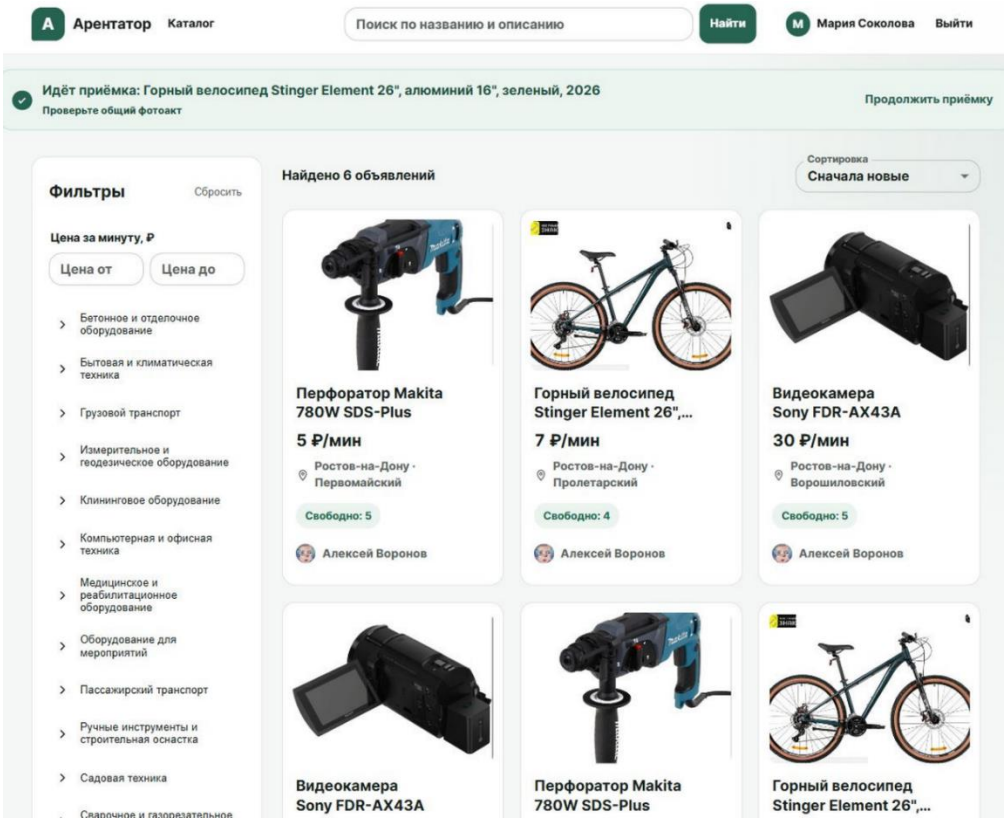


Рисунок 8 – Публичный каталог с поиском и фильтрами

Карточка товара объединяет фотографии, характеристики, расчётную ставку и доступную авторизованному пользователю точку самовывоза.

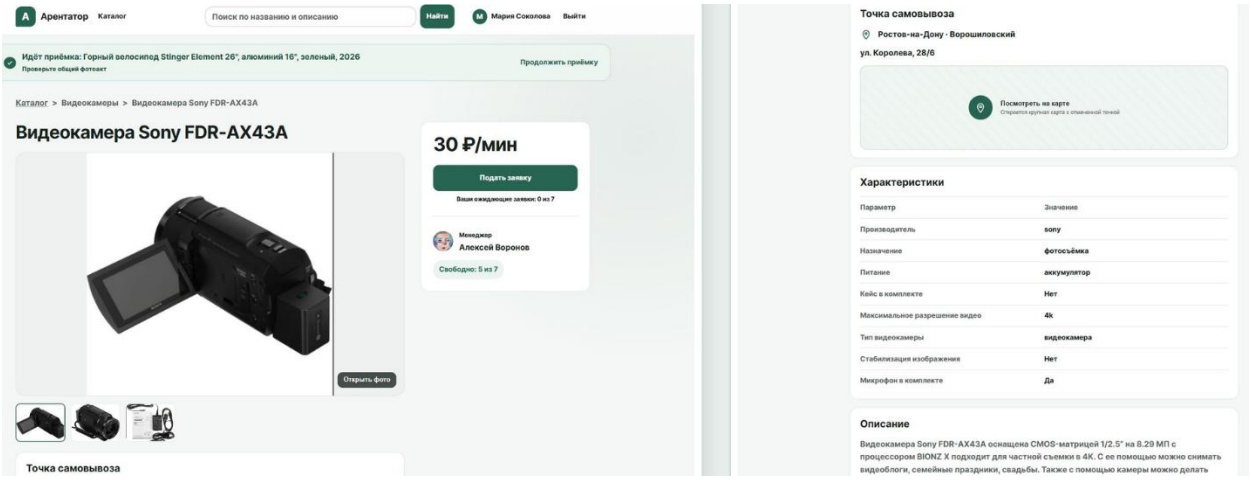


Рисунок 9 – Карточка товара и точка самовывоза

3.4 Реализация заказа

Создание заявки выполняется транзакционно. Сервер блокирует товар, проверяет публикацию и подсчитывает общее число неудалённых экземпляров и ожидающих заявок текущего арендатора. Если лимит достигнут, возвращается структурированная ошибка. Срок получения должен находиться в будущем, а плановый возврат — быть строго позже. Создание сопровождается записью ApplicationEvent и уведомлением менеджера.

При бронировании блокируются заявка и выбранный экземпляр. Повтор той же команды идемпотентно возвращает существующую бронь. Экземпляр должен принадлежать товару заявки, не быть удалённым и иметь статус AVAILABLE. Успешная операция фиксирует снимок ставки и стартовой стоимости, переводит заявку в BOOKED, экземпляр в RESERVED и создаёт события для обеих сторон. Параллельная попытка назначения получает 409 instance_unavailable.

```
@transaction.atomic
def create_booking(*, application_id, instance_id, manager):
    application = (RentalApplication.objects.select_for_update()
        .select_related("product").get(pk=application_id))
    existing = RentalBooking.objects.filter(
        application=application
    ).first()
    if existing is not None:
        if str(existing.instance_id) == str(instance_id):
```

```
                return existing, False
            raise ValidationError({"application": "Бронь уже
создана."})
        instance = ProductInstance.objects.select_for_update().get(
            pk=instance_id
        )
        if instance.status != ProductInstance.Status.AVAILABLE:
            raise InstanceUnavailable()
```

Выдача и возврат представлены отдельными сервисными операциями. Второе подтверждение выдачи создаёт Rental и фиксирует серверное время начала. При возврате receive_return сохраняет return_received_at, после чего формула перестаёт изменяться. Итог учитывает стартовую стоимость десяти минут, своевременные и просроченные минуты, отмену повышающего коэффициента и сумму повреждения. Все денежные значения вычисляются Decimal и округляются до копеек.

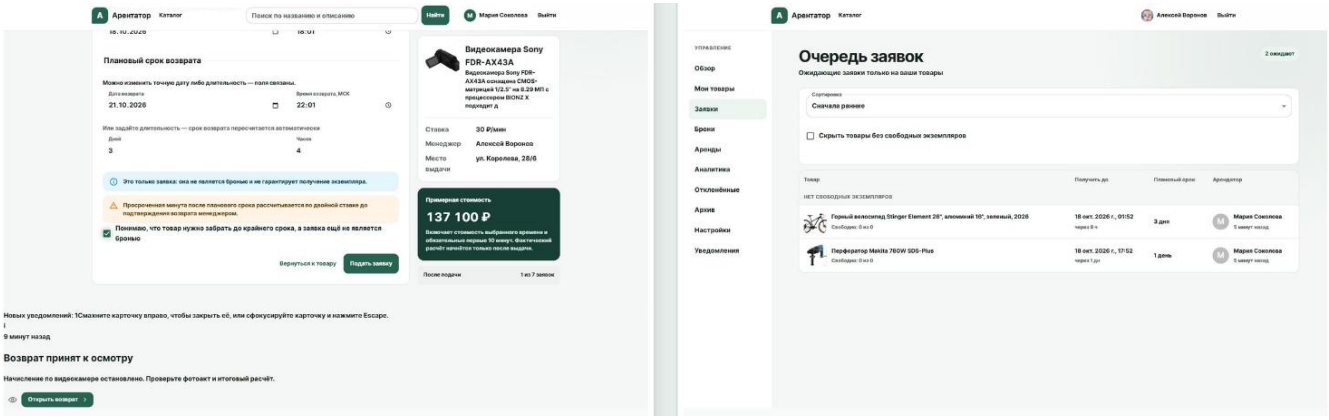


Рисунок 10 – Форма заявки и очередь менеджера

Экран брони показывает назначенный экземпляр, крайний срок получения и последовательность действий до начала оформления выдачи.

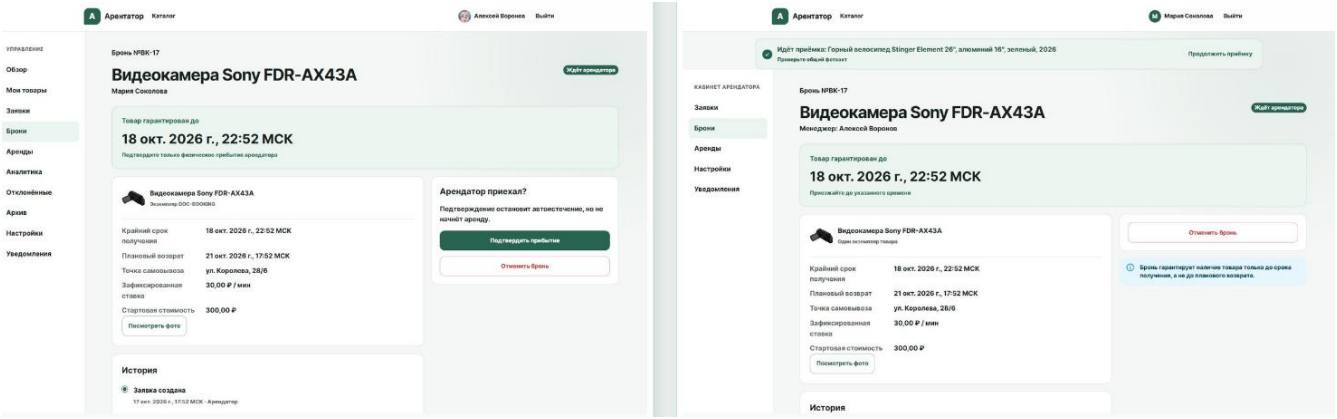


Рисунок 11 – Создание брони и подтверждение прибытия

3.5 Реализация файлов

Загрузка изображений применяется для аватаров, карточек товаров, фотоактов выдачи и возврата, а также контрольного состояния после обслуживания. Backend проверяет размер до 15 МБ, расширение PNG, JPG или JPEG, фактический формат и возможность декодирования. Имя объекта генерируется сервером и не используется как путь напрямую. Ошибка одного файла не отменяет остальные успешно загруженные фотографии.

Публичные изображения и приватные фотоакты разделены. Карточки товаров и аватары могут выдаваться как открытые объекты, а фотоакты доступны только арендатору соответствующей аренды, менеджеру-владельцу и администратору. Для следующего арендатора API выбирает только последний завершённый акт состояния назначенного экземпляра: возврат либо более новый контрольный акт обслуживания. Комментарии, данные прошлого арендатора и инвентарный номер не раскрываются.

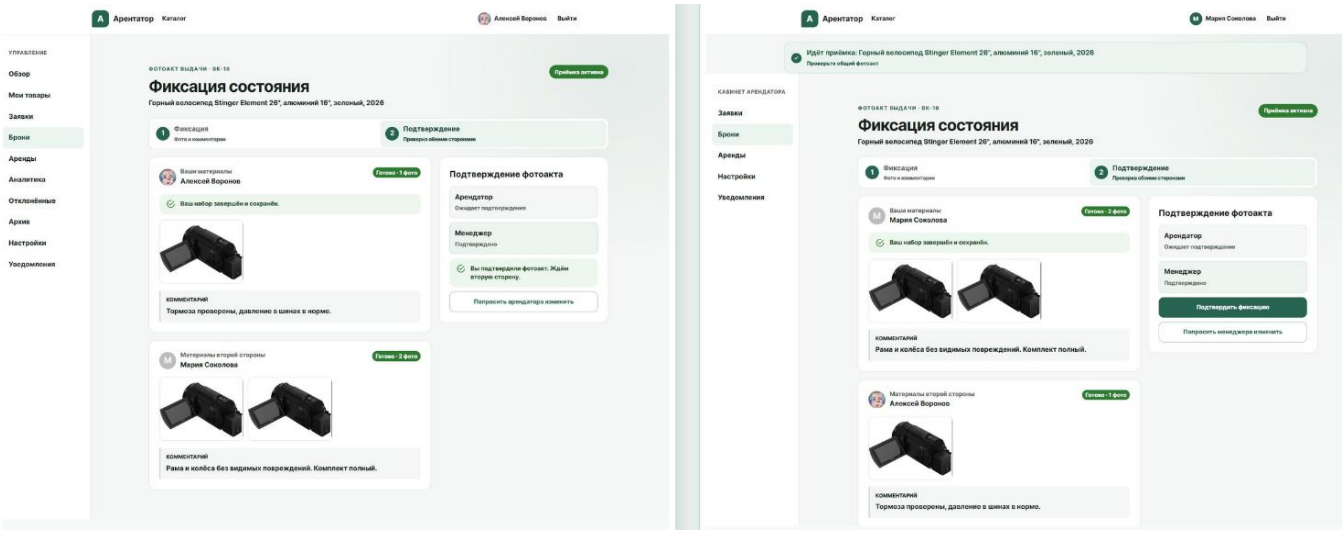


Рисунок 12 – Двусторонний фотоакт выдачи

Экран возврата связывает фотографии обеих сторон, решение по повреждению и итоговый расчёт завершённой аренды.

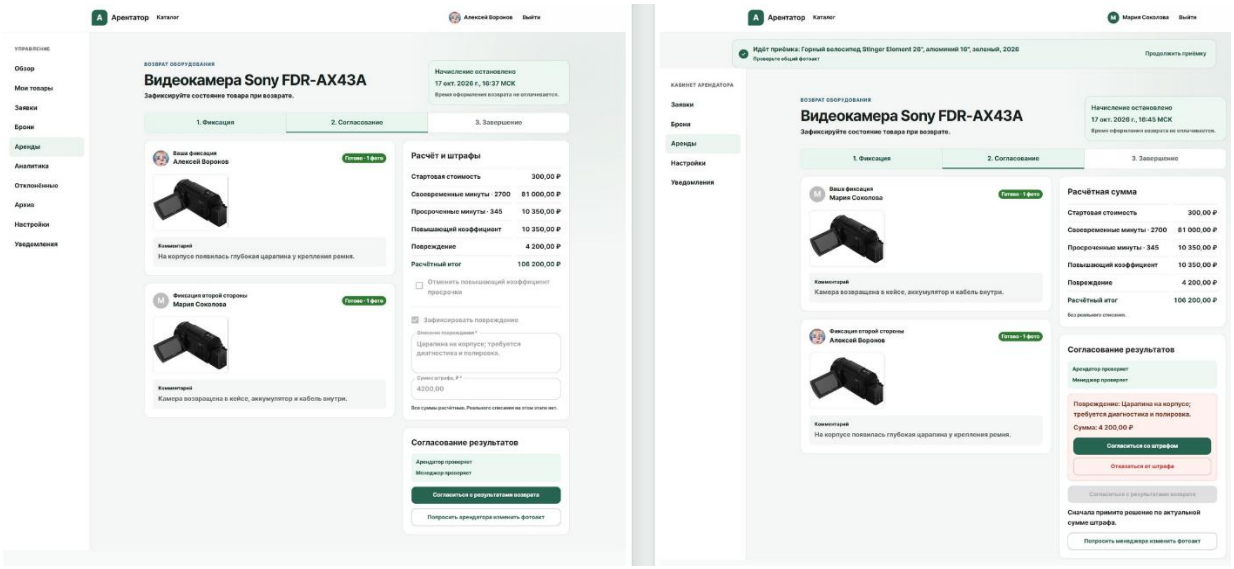


Рисунок 13 – Фотоакт возврата и итоговый расчёт

3.6 Реализация менеджерской части

Кабинет менеджера объединяет товары, заявки, брони, аренды, обслуживание, уведомления, настройки профиля и аналитику. В списке товаров видны состояние модерации, число свободных экземпляров и ожидающих заявок. Редактор разбит на основное описание, фотографии, характеристики, точку самовывоза и экземпляры. Кнопка отправки на модерацию становится доступной после заполнения обязательных характеристик, добавления фото, точки и хотя бы одного экземпляра.

Очередь заявок сначала группируется по наличию свободного экземпляра, затем сортируется по выбранному режиму. Параметры очереди сохраняются для учётной записи. Менеджер явно выбирает экземпляр, подтверждает прибытие, участвует в фотоактах и завершает возврат только после выполнения всех предусловий. Поиск по складу требует полного совпадения инвентарного номера и предлагает только действие, допустимое для текущего состояния.

```
def build_manager_analytics(
    manager: User,
    period: AnalyticsPeriod,
    *,
    now: datetime | None = None,
) -> dict[str, Any]:
    current_time = now or timezone.now()
rentals = _completed_rentals(period, manager=manager)
```

```
        financials = _financial_metrics(rentals, period.end)
        maintenance = _maintenance_metrics(period,
    manager=manager)
        utilization = _utilization_metrics(
            period, manager=manager, now=current_time
        )
        park = _park_metrics(manager=manager)
        overdue = _active_overdue_metrics(
            manager=manager, now=current_time
        )
        category_data = _category_metrics(rentals, period.end)
        total = Decimal(financials["income"]) - Decimal(
            maintenance["expenses"]
        )
        return {
            "period": _period_payload(period),
            "snapshot_at": current_time.astimezone(
                MOSCOW_TIME_ZONE
            ).isoformat(),
            "snapshot_label": _snapshot_label(current_time),
            "financials": {
                **financials,
                **maintenance,
                "net": _money(total),
            },
            "utilization": utilization,
            "active_overdues": overdue,
            "categories": category_data,
        }
```

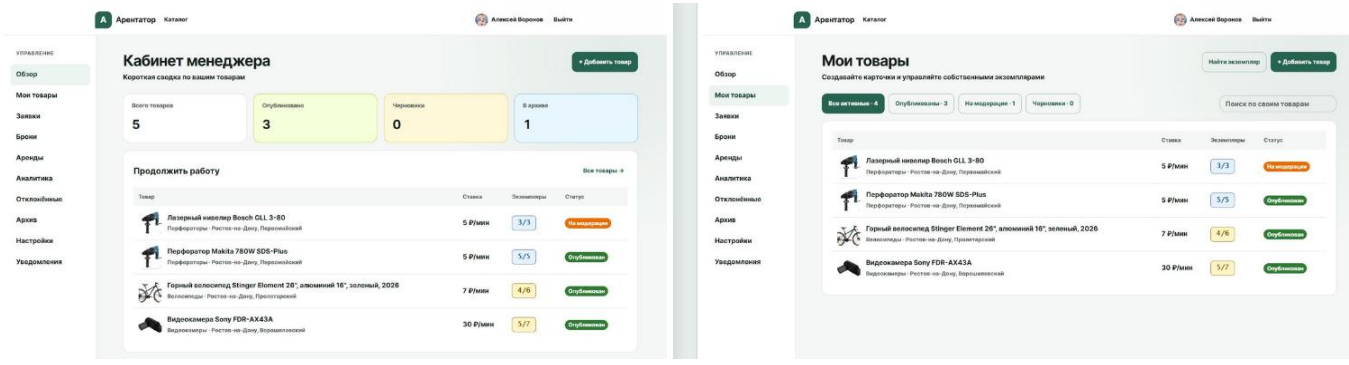


Рисунок 14 – Кабинет менеджера и список товаров

История экземпляра содержит последовательность аренд и обслуживаний, их сроки, причины, затраты и контрольные материалы.

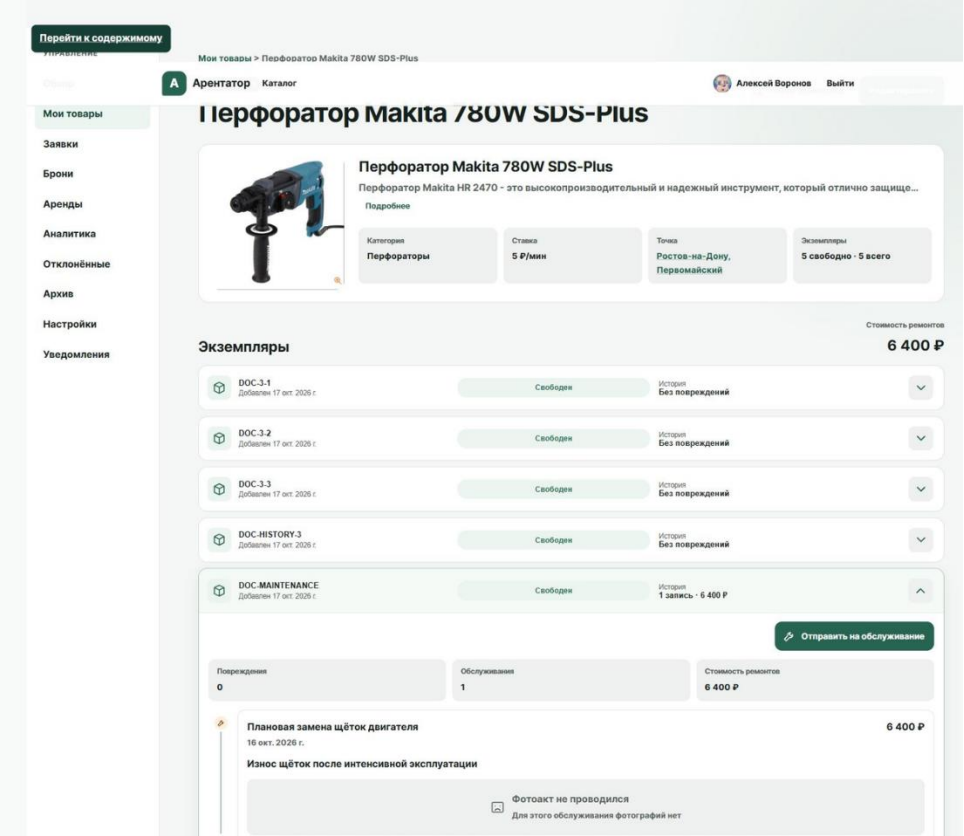


Рисунок 15 – История экземпляра и обслуживание

Аналитика менеджера вынесена в самостоятельную страницу кабинета и формируется только по его товарам и экземплярам. Пользователь выбирает неделю, месяц, год или диапазон дат, включающий обе границы по московскому времени. В верхней части выводятся расчётный доход завершённых аренд с отдельными доплатами за просрочку и повреждения, расходы на завершённое обслуживание, расчётный итог, средняя загрузка экземпляров и количество действующих просроченных аренд.

Ниже расположены столбчатая диаграмма расчётного дохода по категориям, линейная диаграмма загрузки и кольцевая диаграмма текущего состояния парка. Каждая диаграмма содержит период, единицу измерения, прямые значения или легенду и раскрываемую табличную альтернативу. Доходы и расходы обозначены знаками направления, поэтому смысл показателей не зависит только от цвета.

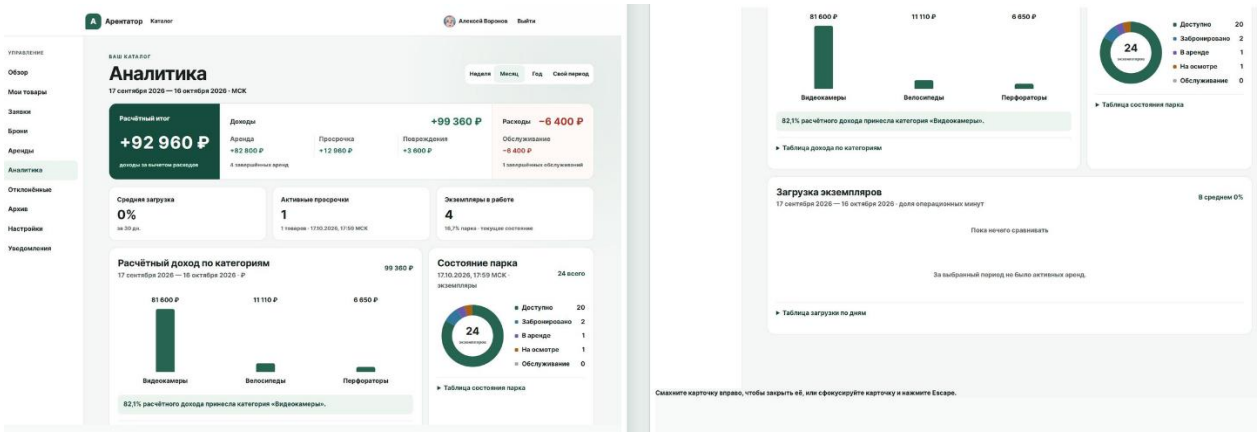


Рисунок 16 – Страница аналитики менеджера

3.7 Реализация администрирования

Административная часть построена на Django admin. Администратор управляет деревом категорий, характеристиками и вариантами, причём форма исключает текущую категорию и её потомков из возможных родителей. Унаследованные названия характеристик передаются в форму и проверяются до отправки; серверная модель сохраняет независимую защиту. Категория с товарами и используемый вариант LIST защищены от удаления.

Модератор видит полную карточку, связанные фотографии, характеристики, точку и экземпляры. Доступны одобрение, отклонение с обязательной причиной, подтверждаемое массовое одобрение, автомодерация новых поступлений, скрытие опубликованного товара и возвращение его в каталог. Каждое решение записывается с автором и признаком автоматического действия, а менеджер получает уведомление о результате.

Аналитика администратора встроена отдельным защищённым маршрутом в пользовательский AdminSite. Она показывает показатели площадки, пользователей, товары, заявки, брони, конверсию и категории за выбранный период. Поле is_active существующего пользователя доступно только для чтения, что соответствует запрету блокировки учётных записей в первой версии.

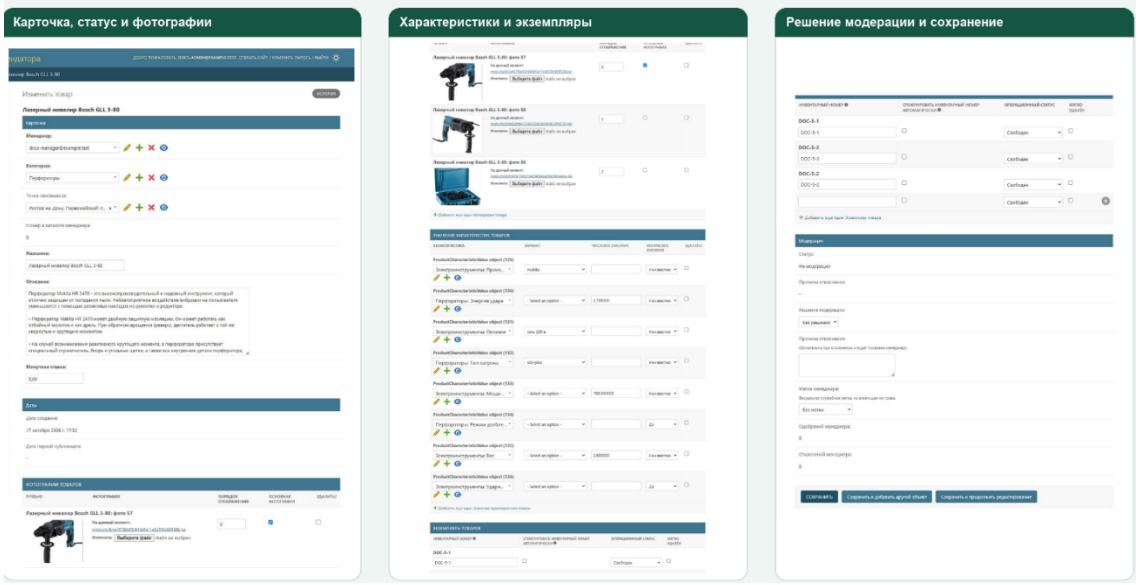


Рисунок 17 – Модерация карточки товара в Django admin

Страница аналитики администратора сохраняет шапку, хлебные крошки и навигацию Django admin, но представляет отдельный сводный отчёт площадки. Она показывает суммарный расчётный доход, расходы на обслуживание, итог, число опубликованных и ожидающих модерации товаров, менеджеров и арендаторов, ожидающих заявок, созданных броней и долю преобразования заявок в брони. Категории сравниваются по расчётному доходу или числу завершённых аренд; периоды рассчитываются по тем же правилам московского времени, что и в кабинете менеджера.

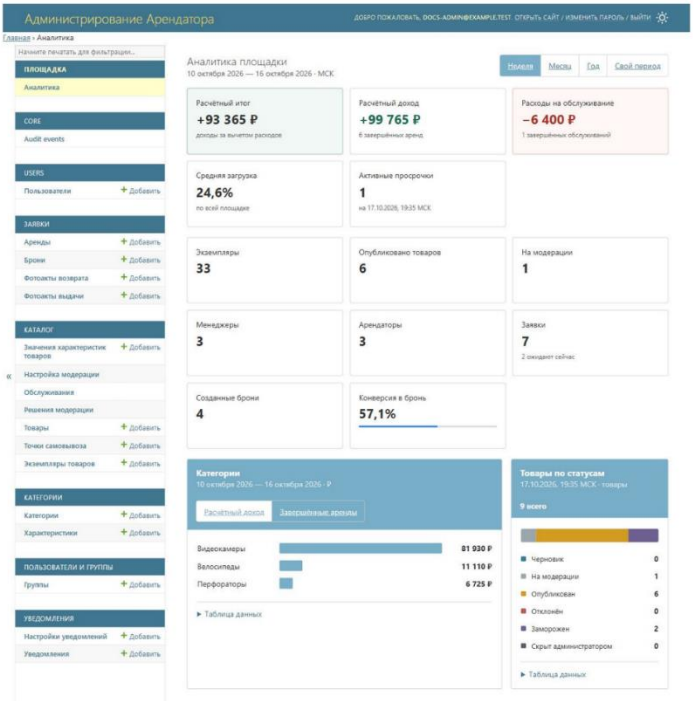


Рисунок 18 – Страница аналитики администратора

3.8 Адаптивность и доступность

Интерфейс спроектирован для контрольных ширин 360, 768 и 1366 px. Каталог изменяет сетку с трёх карточек до одной, страница товара переходит из двух колонок в последовательную компоновку, а рабочие таблицы превращаются в карточки. Временные диаграммы имеют локальную прокрутку. Панели активной приёмки и просрочки не перекрывают основной контент и фокусируемые элементы.

Все поля имеют связанные подписи, а сообщения об ошибках показываются рядом с соответствующим элементом. Критические действия доступны клавиатурой, фокус визуально заметен, размер зон нажатия не меньше 44 px. Статусы сопровождаются текстом и значком, поэтому не зависят только от цвета. Галереи имеют кнопки навигации и обработку клавиш, перетаскивание фотографий дополнено выбором файла и клавиатурной перестановкой.

Центр уведомлений использует доступные имена для иконок информации и предупреждения. Всплывающую карточку можно закрыть жестом или клавишей Escape. Диаграммы имеют текстовые значения и раскрываемые таблицы. Редактор аватара предоставляет одинаковый алгоритм предпросмотра и сохранения; кроме перетаскивания доступны подписанные ползунки масштаба и положения. Для анимаций учитывается prefers-reduced-motion.

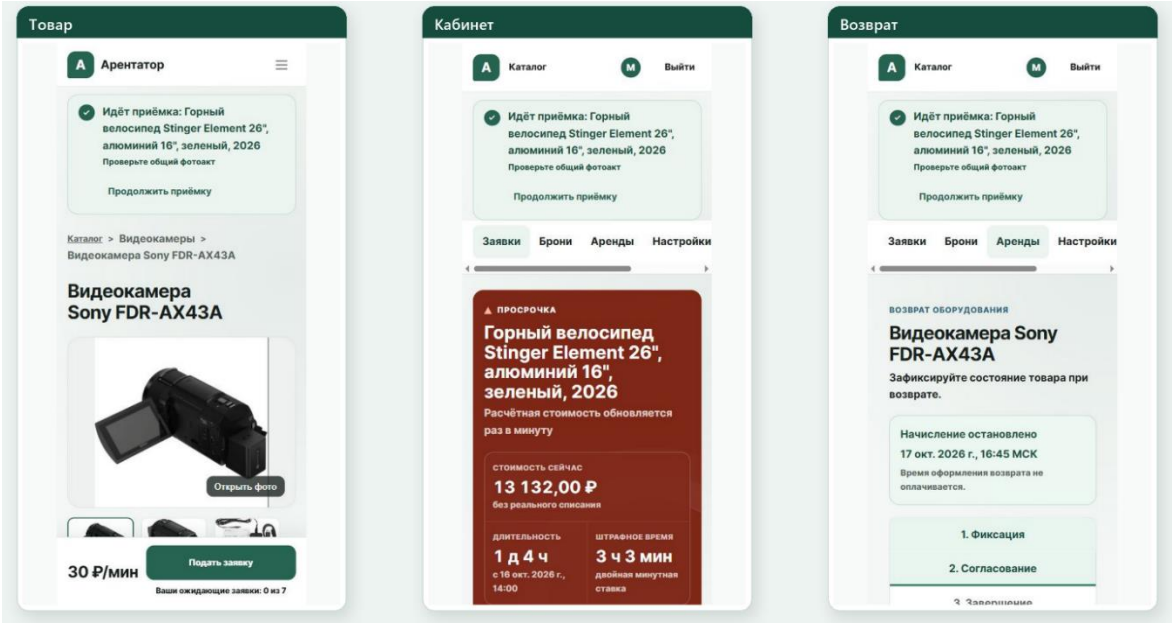


Рисунок 19 – Мобильное представление основных экранов

3.9 Обработка ошибок и логирование

DRF- и CSRF-ошибки приведены к единой структуре. Пользователь получает код, общее сообщение и словарь ошибок полей, поэтому frontend может перенести замечание к конкретному вводу и сохранить остальные значения. Конфликт конкурентного бронирования возвращает 409, отсутствие сессии — 401, запрет роли — 403, а недоступный чужой объект скрывается ответом 404 там, где это требуется безопасностью.

Критические переходы и служебные ошибки записываются backend с достаточным контекстом для диагностики. Пароли, cookie, CSRF-токены, ключи S3, внутренние пути и подписанные ссылки не включаются в пользовательские сообщения и аудит. Неизменяемый AuditEvent создаётся сигналами для предметных сущностей и содержит тип, серверное время, инициатора, роль, безопасное описание и старое или новое состояние при необходимости.

Уведомление сначала сохраняется в базе, затем отправляется после фиксации транзакции. Если Redis или WebSocket временно недоступны, запись остаётся непрочитанной и попадает в офлайн-очередь при следующем подключении. Для менеджера новые заявки и истёкшие брони могут агрегироваться только во всплывающей выдаче, тогда как центр уведомлений сохраняет отдельную запись по каждому объекту.

```
def create_notification(*, recipient, event_type, level, title,
                        message, target_url, target_label):
    settings = NotificationSettings.objects.filter(
        user=recipient
    ).first()
    if settings and event_type in settings.disabled_events:
        return None
    notification = Notification.objects.create(
        recipient=recipient, event_type=event_type, level=level,
        title=title, message=message, target_url=target_url,
        target_label=target_label
    )
    transaction.on_commit(_delivery_callback(notification.pk))
    return notification
```

Страница настроек пользователя доступна из кабинета арендатора и менеджера. На ней можно изменить отображаемое имя, загрузить либо удалить аватар и предварительно настроить квадратное кадрирование изображения. Перемещение фотографии, масштаб и положение доступны как мышью или касанием, так и через подписанные элементы управления. Email и роль показаны как неизменяемые сведения учётной записи.

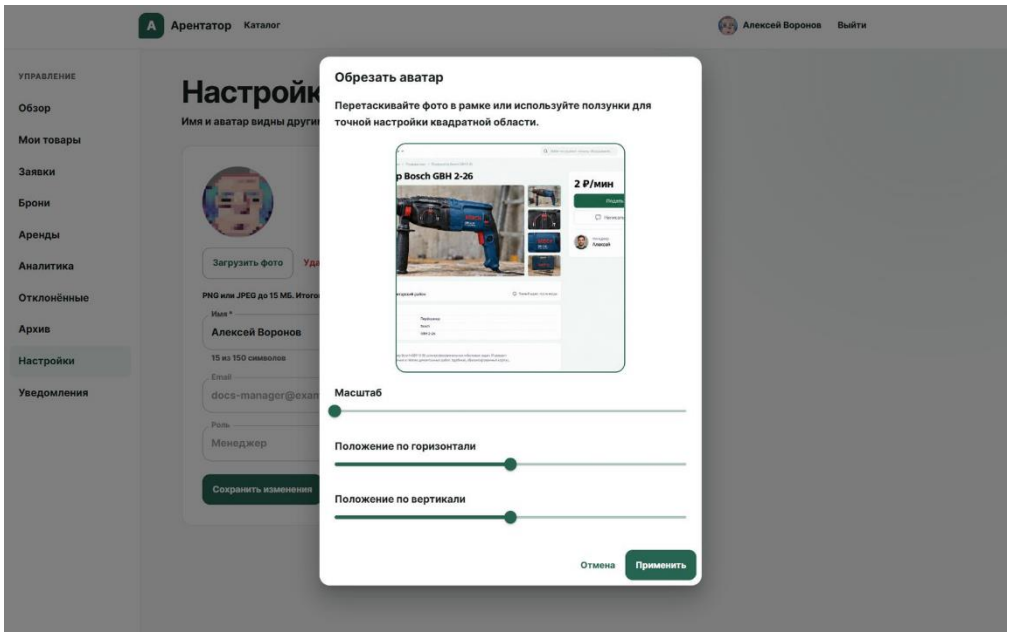


Рисунок 20 – Страница настроек пользователя

Центр уведомлений представляет одноколоночную ленту от новых записей к старым. Для каждой записи указаны смысловая иконка, заголовок, текст, московское время, ссылка на связанный объект и отдельное действие удаления. После трёх секунд непрерывного пребывания на странице записи отмечаются прочитанными. Из заголовка открываются настройки, где для событий роли можно отключить создание будущих уведомлений и отдельно управлять звуком.

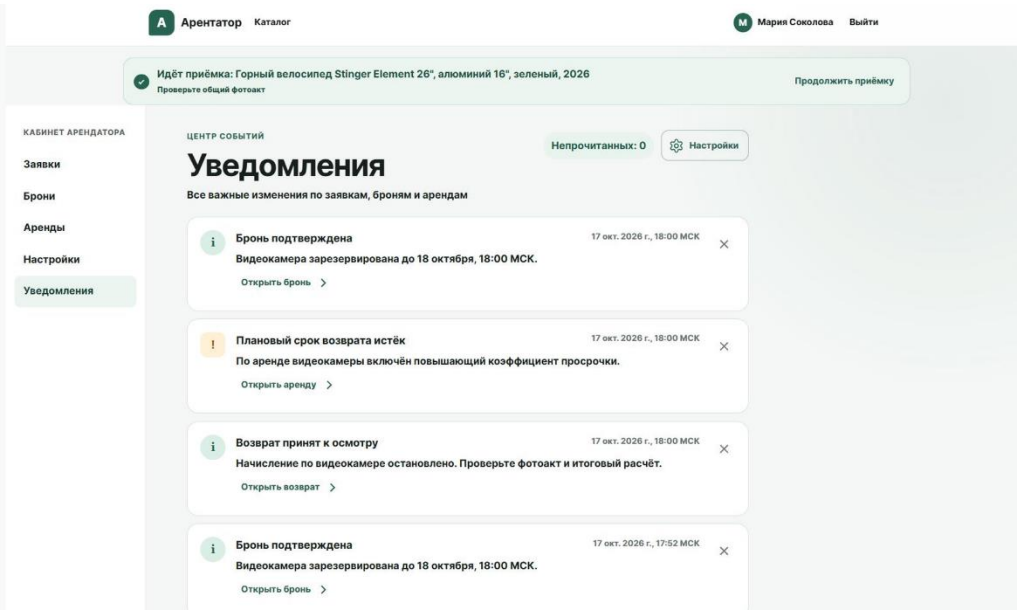


Рисунок 21 – Центр уведомлений

Во время открытой сессии новые записи доставляются через WebSocket и показываются справа снизу в очереди не более чем из трёх всплывающих карточек. Карточка позволяет отметить уведомление прочитанным или перейти к связанному объекту; закрытие жестом либо клавишей Escape не удаляет запись. Для менеджера офлайн-события о новых заявках и истёкших бронях объединяются только во всплывающей выдаче, тогда как центр сохраняет каждое событие отдельно.

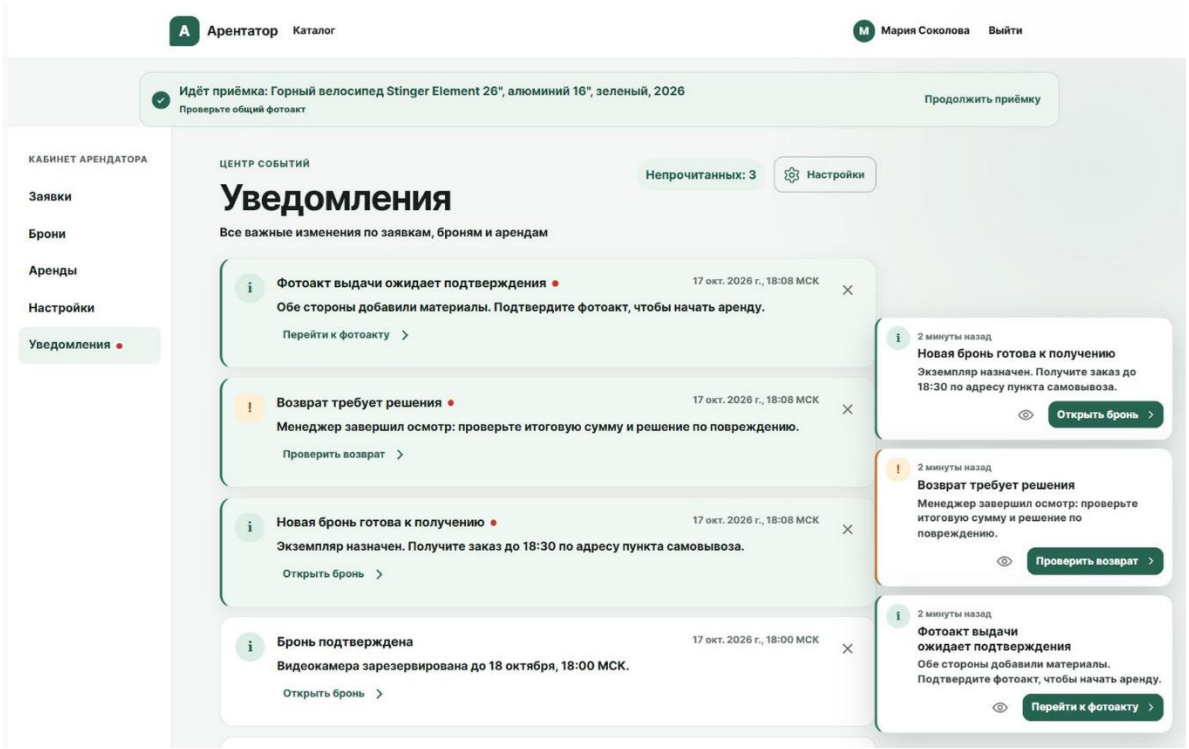


Рисунок 22 – Всплывающие уведомления

4. ТЕСТИРОВАНИЕ И КАЧЕСТВО

4.1 Стратегия тестирования

Стратегия основана на трассировке требований к интерфейсу, API и конкретному тесту. Приёмочная документация содержит восемнадцать последовательных шагов полного сценария: поиск, заявка, бронь, выдача, аренда, просрочка, возврат и освобождение экземпляра. Отдельно выделены тринадцать автоматизированных сценариев ключевой бизнес-логики и девятнадцать негативных проверок. Для каждой строки зафиксированы действие, ожидаемый результат и фактический итог.

Backend-тесты проверяют модели, сериализаторы, транзакционные сервисы, права доступа, истечение сроков, формулы и API. Конкурентное бронирование выполняется на PostgreSQL в двух независимых потоках и подтверждает, что один экземпляр создаёт ровно одну текущую бронь. Frontend-тесты Vitest и Testing Library проверяют пользовательское поведение: маршруты, формы, сообщения полей, видимость действий по роли, фотоакты, уведомления и аналитические состояния.

Статическое качество backend контролируют Black, flake8 и mypy; схема Django проверяется встроенной командой проверки и отсутствием незаписанных миграций. Frontend проходит ESLint, модульные и компонентные тесты, TypeScript и рабочую сборку одной командой npm run check. Браузерные проверки охватывают реальные REST API, две роли, адаптивные ширины и доступность. После исправления каждого дефекта выполняется повторный целевой прогон.

4.2 Таблица тест-кейсов

№	Проверка	Вход/действие	Ожидаемый результат	Результат
1	Поиск и фильтры	Гость вводит запрос, категорию и характеристику	Только подходящие опубликованные товары	Успешно
2	Данные точки для гостя	Открыть карточку без входа	Видны город и район, точный адрес скрыт	Успешно

№	Проверка	Вход/действие	Ожидаемый результат	Результат
3	Данные точки после входа	Войти арендатором	Видны адрес, координаты и карта	Успешно
4	Создание заявки	Будущие pickup_deadline_at и planned_return_at	«Ожидает решения» и запись истории	Успешно
5	Очередь владельца	Менеджер открывает заявки своего товара	Заявка видна только владельцу	Успешно
6	Создание брони	Назначить свободный экземпляр	201, заявка «Забронирована», экземпляр «Забронирован»	Успешно
7	Конкурентная бронь	Два потока назначают один экземпляр	Один 201, один 409, одна бронь	Успешно
8	Прибытие до X	Подтвердить прибытие	бронь «Арендатор прибыл», аренда ещё не начата	Успешно
9	Фото выдачи	Арендатор 2 фото, менеджер 1 фото	Оба набора могут быть завершены	Успешно
10	Начало аренды	Обе стороны подтверждают фотоакт	экземпляр «Арендован» и расчёт после второго подтверждения	Успешно
11	Просрочка	Наступил planned_return_at	аренда «Просрочена» и уведомления обеим сторонам	Успешно
12	Приём возврата	Менеджер фиксирует физический возврат	Начисление прекращается по серверному времени	Успешно
13	Фото возврата	Каждая сторона добавляет минимум одно фото	Наборы завершены и доступны для проверки	Успешно
14	Отмена надбавки	Менеджер отменяет повышающий коэффициент	Поздние минуты считаются по обычной ставке	Успешно
15	Отказ от ущерба	Арендатор отклоняет сумму	Менеджер меняет только сумму, требуется новое согласие	Успешно

№	Проверка	Вход/действие	Ожидаемый результат	Результат
16	Завершение возврата	Есть оба согласия и решение по ущербу	аренда «Завершена», экземпляр «Свободен» или «Обслуживание»	Успешно
17	История и сумма	Открыть завершённую аренду	События и итог совпадают с формулой	Успешно
18	Очередь после освобождения	Экземпляр снова свободен	Заявки показаны по ближайшему X	Успешно
19	Истечение заявки и брони	Установить X в прошлом и выполнить задачу	Сущности истекли, экземпляр освобождён	Успешно
20	Приватность фотоакта	Третий пользователь открывает фото	Доступ отклонён без выдачи данных	Успешно
21	Изоляция менеджеров	Второй менеджер обращается к чужому товару	Чтение и изменение запрещены	Успешно
22	Обслуживание занятого	Экземпляры в состояниях «Забронирован» и «Арендован» направить в обслуживание	Обе операции отклонены	Успешно
23	Файл больше 15 МБ	Загрузить JPEG сверх лимита	Файл не сохранён, показана ошибка	Успешно
24	Подменённый формат	JPEG-расширение и не-графическое содержимое	Файл отклонён	Успешно
25	Цикл категорий	Выбрать потомка родителем предка	Вариант недоступен, цикл не создаётся	Успешно
26	Обязательная характеристика	Отправить товар без значения предка	Модерация отклонена с ошибкой поля	Успешно
27	Категория после публикации	Сменить категорию опубликованного товара	Изменение отклонено	Успешно
28	Точка после публикации	Изменить адрес или pickup point	Изменение отклонено	Успешно

4.3 Найденные и исправленные дефекты

Во время приёмочного прогона обнаружены четыре несоответствия негативным сценариям. Каждый дефект был воспроизведён, локализован, исправлен на соответствующем уровне и закреплён регрессионной проверкой. Исторические сведения сохранены в журнале трассируемости.

Дефект	Причина	Исправление	Повторная проверка
Заявка принимала дату получения в прошлом	Клиентское поле не ограничивало минимальную дату, а API не отклонял прошедший pickup_deadline_at	Добавлено минимальное значение московской даты и серверная ошибка поля pickup_deadline_at	Пройдено: UI блокирует дату, API возвращает 400 validation_error
Возврат мог совпадать со сроком получения	Валидация использовала сравнение < вместо <=	Серверная и клиентская проверки требуют planned_return_at строго позже pickup_deadline_at	Пройдено: равные значения оставляют форму открытой и возвращают ошибку поля
В Django admin можно было создать цикл категорий	Форма предлагала текущую категорию и её потомков как возможного родителя	Недопустимые варианты исключены из списка; модель сохраняет независимую проверку цикла	Пройдено: текущий узел и потомок отсутствуют, цикл не создан
Дочерняя категория принимала характеристику предка с тем же именем	Встроенная форма не проверяла унаследованные названия до сохранения	Добавлены клиентская предварительная проверка и серверная ошибка у поля name	Пройдено: форма показывает ошибку, дубликат не создаётся

Повторный целевой backend-прогон для сроков, циклов и унаследованных характеристик завершился успешно: шесть тестов пройдены. Поведенческий frontend-прогон формы заявки завершился результатом девять тестов из девяти. После повторной проверки новые дефекты в указанных сценариях не обнаружены.

4.4 Проверка безопасности

Проверка безопасности охватывает аутентификацию, авторизацию, CSRF, принадлежность объектов и файлы. Изменяющие запросы без действительного CSRF-токена отклоняются. Завершённая или отсутствующая сессия возвращает 401. Менеджер не может получить товар, заявку или экземпляр другого менеджера подменой идентификатора, а арендатор не имеет доступа к чужой аренде и приватному фотоакту.

Пароли сохраняются средствами Django как стойкие хэши. Cookie-сессия имеет HttpOnly и SameSite, а Secure включается в рабочей конфигурации. Разрешённые источники запросов задаются явно. ORM и параметризованные запросы исключают конструирование SQL из пользовательских строк. React безопасно отображает текст, а API не возвращает трассировки и внутренние пути.

Файлы проверяются по расширению, MIME, фактическому формату, декодированию и размеру. Приватный маршрут фотографии повторно проверяет участника аренды или администратора. Журналы и аудит не содержат паролей, cookie, CSRF-токенов, ключей хранилища и временных ссылок.

4.5 Критерии готовности

Готовность проекта подтверждается полным прохождением основного сценария от каталога до завершённого возврата. Роли и права проверяются сервером, запрещённые переходы отклоняются, финансовая формула и граничные моменты покрыты тестами. Миграции создают схему данных, таксономия загружается повторяемо, а Docker Compose запускает все необходимые сервисы в установленном порядке.

Интерфейс содержит предусмотренные публичные и рабочие экраны, отображает состояния загрузки, отсутствия данных, ошибки, успешного выполнения и запрета доступа, поддерживает клавиатуру и адаптивные ширины. OpenAPI соответствует фактическим маршрутам API. Приёмочная трассировка содержит результаты ручных, автоматизированных и негативных проверок, а журнал дефектов подтверждает их устранение и повторный прогон.

5. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Запуск системы

Для запуска требуется Docker с поддержкой Compose. В корне репозитория файл .env.example копируется в .env. В локальном файле задаются DJANGO_SECRET_KEY, пароль PostgreSQL и данные первоначального администратора; при использовании Yandex Object Storage также указываются параметры S3. Секретные значения не добавляются в Git.

```
mv .env.example .env
docker compose up --build -d
docker compose ps
```

Backend ожидает готовности PostgreSQL и Redis, применяет миграции, загружает таксономию и создаёт администратора, затем запускает ASGI-сервер. После его готовности стартуют frontend, рабочий процесс Celery и Celery Beat. Интерфейс доступен на http://localhost:5173, API — на http://localhost:8000, Swagger UI — на /api/docs/, административная панель — на /admin/. Состояние backend проверяется через /api/health/.

5.2 Работа клиента

1. Откройте каталог, введите запрос, выберите категорию, характеристики, диапазон ставки и сортировку, затем примените фильтры.
2. Откройте карточку товара. Без входа доступны город и район; после входа — полный адрес и карта.
3. Зарегистрируйтесь как арендатор или войдите в существующую учётную запись.
4. Нажмите «Подать заявку», укажите будущий крайний срок получения и более поздний плановый возврат.
5. Следите за состоянием заявки в кабинете. После назначения экземпляра откройте бронь и последние фотографии его состояния.
6. После подтверждения прибытия добавьте не менее двух фотографий выдачи, завершите набор и подтвердите общий фотоакт.

7. Во время аренды контролируйте длительность, плановый возврат и текущую расчётную стоимость.

8. При возврате добавьте минимум одну фотографию, завершите набор, проверьте расчёт и примите решение по сумме повреждения, если она указана.

9. После завершения откройте квитанцию и общую хронологию; уведомления доступны в отдельном разделе кабинета.

5.3 Работа менеджера

После входа менеджер открывает кабинет и создаёт карточку товара. В редакторе указываются название, описание, категория, минутная ставка, фотографии, значения характеристик, точка самовывоза и экземпляры. Полностью заполненная карточка отправляется на модерацию. После публикации менеджер может изменять описание, фотографии, тариф, характеристики и состав экземпляров, а также замораживать предложение.

В очереди заявок выбирается обращение и конкретный свободный экземпляр. После создания брони менеджер подтверждает физическое прибытие арендатора, добавляет собственную фотографию выдачи и участвует в общем подтверждении. Активные и просроченные аренды открываются из отдельного списка. При фактическом возврате менеджер сначала фиксирует его получение, затем добавляет фотографии, при необходимости отменяет надбавку или указывает повреждение и завершает возврат после согласований.

Для обслуживания используется общая страница товара или поиск по точному инвентарному номеру. Свободный экземпляр можно направить в обслуживание с обязательной причиной. При завершении указываются стоимость ремонта и описание, при желании добавляется контрольный фотоакт. Раздел аналитики показывает расчётные доходы, расходы, итог, загрузку и состояние парка за выбранный московский период.

5.4 Работа администратора

Администратор входит в Django admin. В разделе категорий поддерживается дерево, характеристики типов LIST, NUMBER и BOOLEAN и варианты списков. В

очереди модерации открывается карточка со всеми связанными данными; её можно одобрить, отклонить с причиной, скрыть после публикации или вернуть в каталог. Массовое одобрение требует подтверждения, а настройка автомодерации влияет только на новые поступления.

Раздел аудита предназначен для проверки неизменяемой хронологии. Аналитика администратора открывается отдельным маршрутом внутри панели и показывает агрегаты площадки за неделю, месяц, год или произвольный диапазон. Пользователи доступны для просмотра, при этом роли и активность существующей записи защищены правилами первой версии.

5.5 Типовые ошибки пользователя

Сообщение	Причина	Действие
«Неверный email или пароль»	Учётные данные не прошли проверку	Проверить адрес, раскладку и пароль, затем повторить вход
«Укажите срок получения в будущем»	Выбрана прошедшая дата или время	Выбрать будущий московский момент получения
«Плановый возврат должен быть позже»	Возврат совпадает со сроком получения или предшествует ему	Указать более позднюю дату и время возврата
«Выбранный экземпляр уже недоступен»	Другой запрос успел создать бронь	Обновить список и выбрать другой свободный экземпляр
«Размер изображения не должен превышать 15 МБ»	Файл превышает установленный предел	Уменьшить изображение или выбрать другой файл
«Эту аренду нельзя принять к возврату»	Текущее состояние не допускает переход	Проверить статус и выполнить предыдущий обязательный этап

ЗАКЛЮЧЕНИЕ

В результате работы спроектировано и реализовано full-stack web-приложение «Сервис аренды инструмента и оборудования», которое объединяет публичный каталог и полный операционный цикл аренды оборудования. Проведён анализ предметной области, определены роли и границы ответственности, разработаны пользовательские сценарии, архитектура, модель данных и REST API. Реализованы регистрация, профили, категории, динамические характеристики, каталог, поиск, фильтрация, модерация и учёт физических экземпляров.

Система поддерживает заявку, ручное назначение экземпляра, бронь до крайнего срока получения, подтверждение прибытия, двусторонний фотоакт выдачи, фактическую аренду, просрочку, физический возврат, фотоакт возврата и обслуживание. Расчёт использует зафиксированный тариф и точные денежные значения Decimal. Транзакционные операции и ограничения БД предотвращают двойное назначение и недопустимые переходы. Сохраняемые уведомления доставляются через WebSocket, а неизменяемый аудит объединяет историю предметных действий.

Интерфейс реализован на русском языке, адаптирован для мобильной, планшетной и настольной ширины и учитывает клавиатурную навигацию, видимый фокус и текстовое обозначение статусов. Менеджерская и административная аналитика показывают расчётные показатели за выбранный период. Docker Compose обеспечивает воспроизводимый запуск компонентов, миграции и начальную загрузку таксономии.

Приёмочная трассировка подтверждает прохождение сквозного сценария и обязательных негативных проверок. Обнаруженные дефекты сроков и структуры категорий устранены и закреплены регрессионными тестами. Таким образом, поставленная цель достигнута: создана согласованная учебная система, демонстрирующая совместную работу frontend, backend, базы данных, фоновых задач, доставки в реальном времени и контроля качества.

Перечень использованных информационных ресурсов

1. Дронов, В. А. Django 5. Практика создания веб-сайтов на Python. — СПб.: БХВ-Петербург, 2025. — 864 с.
2. Персиваль, Г. Python. Разработка на основе тестирования / пер. с англ. А. В. Логунова. — М.: ДМК Пресс, 2023. — 622 с.
3. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / пер. с англ. — СПб.: Питер, 2024. — 640 с.
4. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / пер. с англ. — СПб.: Питер, 2023. — 352 с.
5. ГОСТ Р ИСО/МЭК 25010—2015. Информационные технологии. Системная и программная инженерия. Требования и оценка качества систем и программного обеспечения. Модели качества систем и программных продуктов.
6. ГОСТ Р 56920—2016. Системная и программная инженерия. Тестирование программного обеспечения. Часть 1. Понятия и определения.
7. Официальная документация языка Python. — URL: https://docs.python.org/ru/3/ (дата обращения: 12.09.2026).
8. Официальная документация Django 6.1. — URL: https://docs.djangoproject.com/en/6.1/ (дата обращения: 14.09.2026).
9. Официальная документация Django REST framework. — URL: https://www.django-rest-framework.org/ (дата обращения: 17.09.2026).
10. Документация PostgreSQL 17.11: официальный перевод на русский язык. — URL: https://postgrespro.ru/docs/postgresql/17/ (дата обращения: 27.09.2026).
11. Официальная документация библиотеки React: русская версия. — URL: https://ru.react.dev/ (дата обращения: 15.10.2026).
12. Документация Yandex Object Storage. — URL: https://yandex.cloud/ru/docs/storage/ (дата обращения: 19.10.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А. РАСПРЕДЕЛЕНИЕ РАБОТЫ В КОМАНДЕ

Участник	Основная зона	Конкретный вклад	Что обязан объяснить на защите

ПРИЛОЖЕНИЕ Б. USER STORIES

– Как гость, я хочу искать оборудование и применять характеристики, чтобы быстро найти подходящее предложение.

– Как арендатор, я хочу видеть точный адрес после входа, чтобы спланировать получение оборудования.

– Как арендатор, я хочу создать заявку даже при отсутствии свободного экземпляра, чтобы ожидать решения менеджера.

– Как арендатор, я хочу видеть последние фотографии назначенного экземпляра, чтобы заранее знать его состояние.

– Как менеджер, я хочу вручную выбрать свободный экземпляр, чтобы контролировать выдачу собственного оборудования.

– Как менеджер, я хочу зафиксировать возврат до осмотра, чтобы время оформления не увеличивало стоимость аренды.

– Как менеджер, я хочу видеть историю ремонтов экземпляра, чтобы оценивать затраты на его эксплуатацию.

– Как администратор, я хочу модерировать карточки и управлять категориями, чтобы поддерживать качество каталога.

ПРИЛОЖЕНИЕ В. МАТРИЦА ПРАВ

Действие	Гость	Арендатор	Менеджер	Админ
Просмотр каталога	Да	Да	Да	Да
Поиск и фильтрация	Да	Да	Да	Да
Просмотр точного адреса	Нет	Да	Да	Да
Создание заявки	Нет	Да	Нет	Нет
Просмотр своих заявок	Нет	Да	Нет	Да
Просмотр заявок на свои товары	Нет	Нет	Да	Да
Назначение экземпляра	Нет	Нет	Да	Нет
Участие в фотоакте	Нет	Да	Да	Просмотр
Приём возврата	Нет	Нет	Да	Нет
Обслуживание экземпляра	Нет	Нет	Да	Просмотр
Управление своими товарами	Нет	Нет	Да	Исправление при модерации
Модерация товаров	Нет	Нет	Нет	Да
Управление категориями	Нет	Нет	Нет	Да
Просмотр аналитики	Нет	Свои операции	Свой каталог	Вся площадка

ПРИЛОЖЕНИЕ Г. README ДЛЯ СДАЧИ

Запуск через Docker Compose

Нужен Docker с поддержкой Compose. Команды ниже выполняются из корня репозитория в Terminal.

1. Создайте локальный файл окружения:

   cp .env.example .env

2. Откройте .env и задайте DJANGO_SUPERUSER_PASSWORD (сложный пароль длиной не менее 8 символов), замените DJANGO_SECRET_KEY на случайную строку. Администратор будет создан с адресом DJANGO_SUPERUSER_EMAIL и именем DJANGO_SUPERUSER_NAME. Файл .env не добавляется в Git. Для production также замените POSTGRES_PASSWORD, настройте домены и CSRF origins, укажите DJANGO_DEBUG=false и DJANGO_COOKIE_SECURE=true.

3. Соберите и запустите сервисы:

   docker compose up --build -d

   docker compose ps

При старте backend дожидается PostgreSQL и Redis, затем последовательно выполняет migrate --noinput, seed_categories и ensure_superuser. Только после их успешного завершения стартует сервер. Celery worker, планировщик и frontend дожидаются работающего backend. Если миграция или заполнение БД завершается с ошибкой, backend не запускается; причину покажет docker compose logs backend. Тот же порядок применяется при запуске backend из production Docker-образа. После запуска доступны:

- frontend: http://localhost:5173;
- health-check backend: http://localhost:8000/api/health/;
- OpenAPI: http://localhost:8000/api/schema/;
- Swagger UI: http://localhost:8000/api/docs/;
- Django admin: http://localhost:8000/admin/;
- дерево категорий: http://localhost:5173/categories.

ПРИЛОЖЕНИЕ Д. ЧЕК-ЛИСТ ПЕРЕД ЗАЩИТОЙ

– репозиторий открывается и содержит последовательную историю разработки;

– проект запускается по README в чистом окружении командой Docker Compose;

– в базе используются демонстрационные записи без реальных персональных данных;

– работают роли арендатора, менеджера и администратора и серверные запреты доступа;

– подготовлен основной сценарий живой демонстрации от каталога до завершённого возврата;

– места для скриншотов в отчёте соответствуют финальным экранам интерфейса;

– ER-диаграмма при заполнении места рисунка соответствует фактическим миграциям;

– тест-кейсы повторно пройдены после последних исправлений;

– презентация сохраняет читаемую структуру и не перегружена текстом;

– докладчик может объяснить архитектуру, модель данных, ролевые ограничения и критические переходы.

ПРИЛОЖЕНИЕ Е. СЦЕНАРИЙ ЖИВОЙ ДЕМОНСТРАЦИИ

Шаг	Действие
1	Открыть публичный каталог, выполнить поиск и применить категорию с характеристикой.
2	Открыть карточку товара без входа и показать только город и район.
3	Войти арендатором и показать полный адрес и встроенную карту.
4	Создать заявку с будущим сроком получения и плановым возвратом.
5	Войти менеджером и открыть заявку на собственный товар.
6	Выбрать свободный экземпляр и создать бронь.
7	Вернуться к арендатору, показать уведомление и прошлое состояние экземпляра.
8	Менеджером подтвердить прибытие до крайнего срока получения.
9	Добавить фотографии выдачи обеими сторонами и завершить наборы.
10	Подтвердить общий фотоакт обеими сторонами и показать начало аренды.
11	Показать таймер, расчётную стоимость и состояние просрочки.
12	Менеджером зафиксировать физический возврат и остановку начисления.
13	Добавить фотографии возврата обеими сторонами.
14	Показать отмену надбавки либо запись о повреждении и расчётную сумму.
15	Продемонстрировать отказ арендатора, изменение только суммы и повторное решение.
16	Подтвердить общий акт и завершить возврат переводом экземпляра в состояние «Свободен» или «Обслуживание».
17	Открыть историю и проверить итоговую формулу завершённой аренды.
18	Показать ожидающие заявки после освобождения экземпляра и аналитику менеджера.